

Optimizing Large Language Models and Blackjack Agent Using Reinforcement Learning Methods

*Report submitted to
Indian Institute of Technology Kharagpur
in partial fulfillment of the requirements for the degree*

of

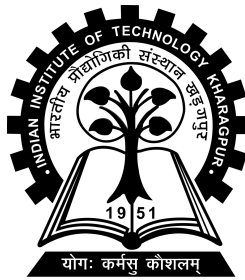
Master of Technology

by

**Adarsh Sharma
(20ME3AI10)**

Under the guidance of

**Prof. Prabhat Kumar Mishra and
Dr. Priyam Chakraborty**



**DEPARTMENT OF ARTIFICIAL INTELLIGENCE
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR**



Department of Artificial Intelligence
Indian Institute of Technology,
Kharagpur
India - 721302

CERTIFICATE

This is to certify that we have examined the thesis entitled **Optimizing Large Language Models and Blackjack Agent Using Reinforcement Learning Methods**, submitted by **Adarsh Sharma** (Roll Number: *20ME3AI10*), an Integrated Dual Degree Program student of the **Department of Artificial Intelligence, IIT Kharagpur**, in partial fulfillment of the requirements for the award of the degree of Master of Technology in Artificial Intelligence.

We hereby accord our approval of the thesis as a study carried out and presented in a manner required for its acceptance in partial fulfillment of the postgraduate degree for which it has been submitted. The thesis has fulfilled all the requirements as per the regulations of the Institute and meets the standard necessary for submission.

**Prabhat Kumar
Mishra**

**Department of Artificial
Intelligence**
Indian Institute of Technology,
Kharagpur

Place: Kharagpur
Date: May 4, 2025

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to the individuals and institutions who have been instrumental in the successful completion of this thesis.

First and foremost, I am profoundly thankful to my thesis advisors, Professor Prabhat Kumar Mishra and Dr. Priyam Chakraborty, for their unwavering guidance, patience, and invaluable insights throughout the research process. Their expertise and mentorship have been pivotal in shaping the direction and quality of this work.

I would also like to acknowledge the Indian Institute of Technology, Kharagpur, for providing the necessary resources and facilities to conduct this research. The institution's academic environment has greatly fostered my growth and learning.

Additionally, I am grateful to my friends and colleagues for their moral support, thoughtful discussions, and camaraderie, which made this research journey both enriching and enjoyable.

This thesis would not have been possible without the collective contributions and support of these remarkable individuals and institutions. I am sincerely thankful for their involvement and encouragement throughout this academic endeavor.

Adarsh Sharma

IIT Kharagpur

Date: April 27, 2024

Abstract

This thesis investigates the application of Reinforcement Learning (RL) across two distinct environments: large language models (LLMs) and the game of Blackjack. For the LLM setting, we employ Reinforcement Learning from Human Feedback (RLHF) to enhance performance on text summarization tasks. Using the OpenAI *Summarize from Feedback* dataset, we fine-tuned both reward and supervised models, and implemented Proximal Policy Optimization (PPO) and Direct Preference Optimization (DPO). To manage the computational challenges associated with large models (up to 7 billion parameters), techniques such as gradient accumulation and model quantization were utilized. We also explored alternative RL algorithms (A2C, A3C, TRPO), highlighting the advantages of PPO in terms of stability and performance.

In the Blackjack environment, we applied classical Monte Carlo methods and policy gradient methods, with a focus on Proximal Policy Optimization (PPO) and a newer RL algorithm named Group Relative Policy Optimization (GRPO). While PPO was implemented directly by using the Stable Baselines library, GRPO was implemented in python. Integrating card-counting features enabled more informed betting strategies, resulting in an average reward per bet of 2.36%. Additionally, we implemented an extended GRPO algorithm, Decoupled Clipping and Dynamic sAmpling Policy Optimization (DAPO), to promote exploration. The DAPO-enhanced agent demonstrated improved exploratory behavior and achieved a higher win rate (44.39%) compared to standard PPO.

These findings illustrate the efficacy of reinforcement learning in both optimizing language model behavior and enhancing strategic decision-making in complex environments.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Research Objectives	2
2	Literature Review	3
2.1	Reinforcement Learning Methods	3
2.1.1	Classical Value-Based RL Algorithms	3
2.1.1.1	Monte Carlo Method	3
2.1.1.2	Temporal Difference (TD) Learning	3
2.1.1.3	Q-Learning	3
2.1.1.4	SARSA	4
2.1.1.5	Deep Q-Learning (DQN)	4
2.1.1.6	Comparison of Value-Based Methods	4
2.1.2	Policy Gradient RL Algorithms	5
2.1.2.1	REINFORCE	5
2.1.2.2	Advantage Actor-Critic (A2C/A3C)	5
2.1.2.3	Deep Deterministic Policy Gradient (DDPG)	5
2.1.2.4	Trust Region Policy Optimization (TRPO)	5
2.1.2.5	Proximal Policy Optimization (PPO)	6
2.1.2.6	Comparison of Policy Gradient Methods	6
2.1.3	Group Relative Policy Optimization	6
2.1.3.1	Objective Function	7
2.1.3.2	Training Loop	7
2.1.3.3	Key Hyperparameter for GRPO	7
2.2	Training LLMs on Custom Datasets	8
2.3	Training Pipeline and Impact of RL (GRPO) on DeepSeek R1 Training	9

2.3.1	DeepSeek R1-Zero Evaluation	10
2.3.2	DeepSeek R1 Evaluation	11
3	Methodology	12
3.1	Text Generation and Summarization Methodology	12
3.1.1	Improving Text Generation with RL	12
3.1.2	RLHF Architecture: Why PPO and Not A2C?	13
3.1.3	Our Task: Text Summarization	14
3.1.4	Dataset: OpenAI/SummarizeFromFeedback	14
3.1.4.1	Dataset Overview	14
3.1.5	Models	15
3.1.5.1	RL Algorithms Considered	15
3.1.5.2	Why PPO?	16
3.1.5.3	Computational Challenges and Optimization Strategies	16
3.1.5.4	Use of Smaller Models for Reward Classification . . .	16
3.1.5.5	Direct Preference Optimization (DPO)	16
3.2	Blackjack Environment	17
3.2.1	Game Setup	17
3.2.2	Player Actions	17
3.2.3	Dealer’s Turn	17
3.2.4	Card Counting and Strategy	17
3.2.5	Natural Blackjack	17
3.2.6	Types of Blackjack Environments	18
3.2.6.1	Simple Blackjack Environment	18
3.2.6.2	Extended Blackjack Environment	18
3.2.6.3	Extended Blackjack Environment with Card Counting	18
3.2.7	Evaluation Metrics for Blackjack	18
3.2.8	Training Setup	18
3.2.8.1	Simple Blackjack Environment	18
3.2.8.2	Extended Blackjack Environment	19
3.2.8.3	Extended Blackjack Environment with Card Counting	19

4	Results and Discussion	20
4.1	LLM Experiments	20
4.1.1	Training RLHF Model with GPT-2	20
4.1.2	Gemma-2B vs. Gemma-7B on Text Classification	20
4.1.3	Fine-Tuning Gemma-2B Reward Model & LLaMA-3.1 8B	21
4.2	Results for the Blackjack Environment	21
4.2.1	Simple Blackjack Environment	21
4.2.1.1	Monte Carlo Method	22
4.2.1.2	Proximal Policy Optimization (PPO)	23
4.2.2	Extended Blackjack Environment	25
4.2.2.1	Monte Carlo Method	25
4.2.2.2	Proximal Policy Optimization (PPO)	28
4.2.2.3	Important Insights	29
4.2.3	Extended Blackjack Environment with Card Counting	29
4.2.3.1	Why Deep RL-Based Methods Are Required?	30
4.2.3.2	Proximal Policy Optimization (PPO)	30
4.2.3.3	Group Relative Policy Optimization Training (GRPO)	33
4.2.3.4	Saving the Trained Models	36
5	Web Application for the Extended Blackjack Environment with Card Counting	37
5.1	User Workflow	38
5.2	Implementation Details	40
6	Conclusion	41
7	Future Work	43

List of Figures

2.1	Key hyperparameter for GRPO	8
2.2	Performance evaluation of DeepSeek R1-Zero.	10
2.3	Performance evaluation of DeepSeek R1.	11
3.1	RLHF architecture for generative text generation	13
3.2	Example schema from the OpenAI SummarizeFromFeedback dataset	15
4.1	Average reward per bet during training of the Monte Carlo agent. . .	22
4.2	Evaluation metrics for the Monte Carlo agent after training.	23
4.3	Learned strategy heatmap (policy chart) for the Monte Carlo agent in the simple environment.	23
4.4	Evaluation metrics for the PPO agent in the simple Blackjack environ- ment.	24
4.5	Average reward per bet during training of the Monte Carlo agent in the extended environment.	26
4.6	Evaluation metrics for the Monte Carlo agent in the extended environ- ment.	26
4.7	Learned strategy heatmap for the Monte Carlo agent in the extended environment.	27
4.8	Average reward per bet during training of the PPO agent in the ex- tended environment.	28
4.9	Evaluation metrics for the PPO agent in the extended environment. .	28
4.10	Phase 1: Avg. reward per bet during PPO training (10 M timesteps).	32
4.11	Phase 1: Evaluation metrics after 10 M timesteps.	32
4.12	Phase 2: Avg. reward per bet during PPO training (+20 M timesteps).	33
4.13	Phase 2: Evaluation metrics after total of 30 M timesteps.	33
4.14	Reward distributions over 100 000 episodes: PPO (left) vs. GRPO (right).	34

4.15	Reward distribution for DAPO-enhanced GRPO (<code>drpo_distribution</code>).	35
4.16	Average reward per bet during training of the final GRPO(DAPO) agent in the extended environment.	36
4.17	Evaluation metrics for the final GRPO(DAPO) agent in the extended environment.	36
5.1	Flow diagram.	38
5.2	Modal for selecting dealer and player cards.	39
5.3	Main interface displaying game state and advice.	39
5.4	“Lost game” message when the player busts.	40

Chapter 1

Introduction

1.1 Background and Motivation

Reinforcement learning (RL) is a paradigm of machine learning in which an agent interacts with an environment by taking actions and receiving feedback in the form of rewards, with the goal of learning a policy that maximizes cumulative reward over time. The use of reinforcement learning across various fields has increased steadily; it can be applied to a wide range of environments. The impact of RL is exemplified by OpenAI’s GPT-3.5 model, introduced in early 2022, which only became a widely recognized phenomenon when the model was fine-tuned with reinforcement learning from human feedback (RLHF) to create ChatGPT, delivering remarkably human-like responses. More recently, the DeepSeek-R1 model was released(3), achieving performance on numerous reasoning metrics that closely rival OpenAI’s flagship O1 model. Reinforcement learning—specifically the GRPO algorithm—played an integral role in the DeepSeek-R1 training pipeline. DeepSeek-R1 uses a 671-billion-parameter base model named DeepSeek-V3, which is considerably smaller than O1, yet its reasoning ability is nearly equivalent, showcasing the transformative impact reinforcement learning can have. Additionally, DeepSeek integrates advanced techniques such as Multi-Head Latent Attention (MHLA) (7) and Mixture of Experts (MoE) (8) to enhance performance while optimizing memory efficiency. MHLA allows the model to selectively focus on latent representations across multiple attention heads, reducing redundant computation and improving contextual understanding with lower memory overhead. Meanwhile, the MoE architecture enables the model to activate only a subset of specialized expert networks per input, significantly cutting down on computational cost without sacrificing performance. Notably, the distilled versions of

DeepSeek have also demonstrated impressive capabilities, achieving competitive results with reduced model size and inference time—making them highly efficient for deployment in resource-constrained environments.

1.2 Research Objectives

This thesis investigates the impact of various RL algorithms on training agents in two environments: Blackjack and large language models (LLMs). In both cases, the reward signal is non-zero only at the terminal step of each episode. In the Blackjack environment, episodes typically consist of a limited number of steps, often fewer than ten (e.g., HHHHHS, where H stands for *hit* and S for *stand*), whereas in generative tasks such as text summarization, an LLM may produce trajectories comprising dozens of tokens. This disparity poses significant challenges in designing RL pipelines.

Since no prior implementation of the GRPO algorithm on a Blackjack dataset exists, we adapted the GRPO pipeline originally proposed for LLM fine-tuning(17) to the Blackjack environment and trained the agent accordingly. The GRPO paper’s pipeline updates the policy only at the end of each episode, and we adopted the same approach. Our results demonstrate that the GRPO-trained agent outperforms a popular PPO implementation in terms of win percentage but yields a lower average reward per bet. The PPO implementation, taken from Stable Baselines(1), performs step-wise updates. In future work, we plan to enhance our GRPO pipeline by incorporating step-wise updates to improve average reward performance.

Chapter 2

Literature Review

2.1 Reinforcement Learning Methods

2.1.1 Classical Value-Based RL Algorithms

2.1.1.1 Monte Carlo Method

The Monte Carlo method updates state-value estimates using full-episode returns (9):

$$v^{\text{new}}(s) = v^{\text{old}}(s) + \alpha(G_t - v^{\text{old}}(s)) \quad (2.1)$$

Explanation: Here, G_t is the cumulative reward obtained from time t until the end of the episode. Monte Carlo methods require complete episodes before updating and provide unbiased but high-variance estimates of the true value function.

2.1.1.2 Temporal Difference (TD) Learning

TD(0) combines ideas from Monte Carlo and dynamic programming by bootstrapping (9):

$$v^{\text{new}}(s_t) = v^{\text{old}}(s_t) + \alpha(R_{t+1} + \gamma v^{\text{old}}(s_{t+1}) - v^{\text{old}}(s_t)). \quad (2.2)$$

Explanation: TD learning updates estimates after each step using the one-step reward R_{t+1} plus the discounted estimate of the next state. This reduces variance and allows online updates without waiting for episode termination.

2.1.1.3 Q-Learning

An off-policy action-value method (10):

$$Q^{\text{new}}(s_t, a_t) = Q^{\text{old}}(s_t, a_t) + \alpha\left(R_{t+1} + \gamma \max_a Q^{\text{old}}(s_{t+1}, a) - Q^{\text{old}}(s_t, a_t)\right). \quad (2.3)$$

Explanation: Q-Learning learns the optimal action-value function irrespective of the policy being followed. By taking the maximum over next-state values, it converges to the optimal policy in the limit.

2.1.1.4 SARSA

An on-policy action-value method (10):

$$Q^{\text{new}}(s_t, a_t) = Q^{\text{old}}(s_t, a_t) + \alpha \left(R_{t+1} + \gamma Q^{\text{old}}(s_{t+1}, a_{t+1}) - Q^{\text{old}}(s_t, a_t) \right). \quad (2.4)$$

Explanation: SARSA updates using the action actually taken in the next state (on-policy), leading to safer learning behavior but potentially slower convergence than off-policy methods.

2.1.1.5 Deep Q-Learning (DQN)

DQN approximates $Q(s, a)$ with a neural network $Q_\theta(s, a)$ and uses experience replay and a target network for stability (11):

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a) \right)^2 \right]. \quad (2.5)$$

Explanation: By sampling mini-batches from a replay buffer $\mathcal{D}$ and periodically updating a frozen target network Q_{θ^-} , DQN mitigates correlated updates and non-stationary targets, enabling stable training of value-based methods with deep function approximators.

2.1.1.6 Comparison of Value-Based Methods

Method	Policy	Strength	Weakness
Monte Carlo (MC)	On-policy / Off-policy	Unbiased estimates	High variance; needs full episode
TD Learning (TD(0))	On-policy	Fast, online updates	Some bias; approximation errors
Q-Learning	Off-policy	Learns optimal policy directly	Can overestimate; unstable
SARSA	On-policy	Safer learning; more stable	Slower convergence; more conservative
Deep Q-Learning (DQN)	Off-policy	Handles large/complex spaces	Requires replay buffer and target network

Table 2.1: Comparison of classical value-based RL algorithms by policy, strengths, and weaknesses.

2.1.2 Policy Gradient RL Algorithms

2.1.2.1 REINFORCE

The basic Monte Carlo policy-gradient method updates policy parameters using full-episode returns (12):

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t. \quad (2.6)$$

Explanation: REINFORCE computes gradients using the log-probability of taken actions weighted by the total return G_t . It is simple and unbiased but suffers from high variance and requires complete episodes.

2.1.2.2 Advantage Actor–Critic (A2C/A3C)

Updates combine policy gradients with a learned value-function critic (13):

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A(s_t, a_t), \quad A(s_t, a_t) = Q(s_t, a_t) - V(s_t). \quad (2.7)$$

Explanation: By subtracting a baseline $V(s)$, A2C/A3C reduces variance in the gradient estimate. A3C uses multiple asynchronous workers to further stabilize and parallelize learning.

2.1.2.3 Deep Deterministic Policy Gradient (DDPG)

A method for continuous action spaces using off-policy updates (14):

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim \mathcal{D}} [\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q^{\pi}(s, a)]. \quad (2.8)$$

Explanation: DDPG employs a deterministic policy (actor) and a critic network, along with experience replay and target networks, to learn in continuous action domains. It is sample-efficient but can be unstable and sensitive to hyperparameters.

2.1.2.4 Trust Region Policy Optimization (TRPO)

Policy updates constrained by a KL-divergence trust region (15):

$$\max_{\theta} \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a | s) A(s, a)] \quad \text{s.t.} \quad D_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_{\theta}) \leq \delta. \quad (2.9)$$

Explanation: TRPO ensures each update stays within a trust region to guarantee monotonic policy improvement. It is stable but computationally expensive due to the KL-constraint optimization.

2.1.2.5 Proximal Policy Optimization (PPO)

A simpler, clipped surrogate objective that approximates TRPO (16):

$$L_{\text{CLIP}}(\theta) = \mathbb{E} \left[\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t) \right], \quad r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}. \quad (2.10)$$

Explanation: PPO uses a clipped probability ratio to limit policy updates, achieving stable learning similar to TRPO but with simpler implementation and lower computational cost.

2.1.2.6 Comparison of Policy Gradient Methods

Algorithm	Policy	Strength	Weakness
REINFORCE	On-Policy	Unbiased gradient estimate	High variance; slow convergence
A2C/A3C	On-Policy	Lower variance via critic; parallelizable	Requires value-function tuning
DDPG	Off-Policy	Handles continuous actions; sample-efficient	Unstable; sensitive to hyperparameters
TRPO	On-Policy	Guaranteed policy improvement; stable updates	Computationally expensive
PPO	On-Policy	Stable and efficient; easy to implement	Still needs careful hyperparameter selection

Table 2.2: Comparison of policy gradient RL algorithms by policy, strengths, and weaknesses.

2.1.3 Group Relative Policy Optimization

Mathematical reasoning remains one of the most challenging capabilities for open large language models (LLMs), due to the need for precise symbol manipulation, long-range dependencies, and structured problem solving. In February 2024, Shao *et al.* introduced DeepSeekMath 7B, a model that extended pre-training on 120 billion math-related tokens and demonstrated state-of-the-art performance on the MATH benchmark without external tools. A central innovation of DeepSeekMath was Group Relative Policy Optimization (GRPO), a novel variant of Proximal Policy Optimization (PPO) designed to improve reasoning accuracy and computational efficiency.

GRPO was formally presented in the DeepSeekMath paper in April 2024 (17). Unlike standard PPO, which computes policy updates at the individual token or step level, GRPO groups G generated tokens (or reasoning steps) into mini-batches and optimizes a single surrogate objective over each group. This grouped approach yields two major benefits:

- **Stable Credit Assignment:** By normalizing advantages across G related outputs, GRPO reduces gradient variance and avoids noisy updates that can derail multi-step reasoning.
- **Memory and Compute Efficiency:** Without requiring a separate critic network, GRPO achieves comparable stability to actor–critic methods using only a single policy network, thereby halving memory usage and reducing per-update overhead.

In DeepSeekMath, GRPO was applied to both intermediate pre-training and downstream fine-tuning stages. The authors reported a jump from 49.3% to 51.7% accuracy on the MATH benchmark using DeepSeekMath 7B, and a further gain to 60.9% under self-consistency sampling. These results approached the performance of much larger proprietary models such as GPT-4 and Gemini-Ultra, highlighting the practical impact of GRPO for open models.

Beyond mathematical tasks, GRPO has since been adopted in the DeepSeek-R1 project (January 2025)(3), where it underpins advanced “aha-moment” self-correction behaviors by dynamically allocating extra computation to difficult problems. It has also inspired adaptations in multimodal reasoning systems and strategic game agents, demonstrating its versatility across domains.

2.1.3.1 Objective Function

The GRPO loss combines a clipped surrogate objective over a group of G outputs with a KL-divergence regularizer against a reference policy:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q \sim \mathcal{Q}} \left[\frac{1}{G} \sum_{i=1}^G \min(r_i(\theta) A_i, \text{clip}(r_i(\theta), 1 - \epsilon, 1 + \epsilon) A_i) \right] - \beta D_{\text{KL}}(\pi_{\text{old}} \parallel \pi_{\text{ref}}), \quad (2.11)$$

where

$$r_i(\theta) = \frac{\pi_{\theta}(o_i \mid q)}{\pi_{\text{old}}(o_i \mid q)}, \quad A_i = \frac{r_i - \bar{r}}{\text{std}(r_1, \dots, r_G)}.$$

2.1.3.2 Training Loop

2.1.3.3 Key Hyperparameter for GRPO

Algorithm 1 Iterative Group Relative Policy Optimization

Require: initial policy model $\pi_{\theta_{\text{init}}}$, reward models r_{ϕ} , task prompts $\mathcal{D}$, hyperparameters ϵ, β, μ

```
1: policy model  $\pi_{\theta} \leftarrow \pi_{\theta_{\text{init}}}$ 
2: for iteration = 1, ...,  $I$  do
3:   reference model  $\pi_{\text{ref}} \leftarrow \pi_{\theta}$ 
4:   for step = 1, ...,  $M$  do
5:     Sample a batch  $\mathcal{D}_b$  from  $\mathcal{D}$ 
6:     Update the old policy model  $\pi_{\text{old}} \leftarrow \pi_{\theta}$ 
7:     Sample  $G$  outputs  $\{o_i\}_{i=1}^G \sim \pi_{\text{old}}(\cdot | q)$  for each  $q \in \mathcal{D}_b$ 
8:     Compute rewards  $\{r_i\}_{i=1}^G$  for each  $o_i$  by running  $r_{\phi}(o_i)$ 
9:     Compute  $\hat{A}_i$  for the  $t$ -th token of  $o_i$  via group relative advantage estimation
10:    for GRPO iteration = 1, ...,  $\mu$  do
11:      Update policy model  $\pi_{\theta}$  by maximizing the GRPO objective (Eq. 2.11)
12:    end for
13:    Update  $r_{\phi}$  through continuous training using a replay mechanism
14:  end for
15: end for
```

Ensure: Optimized policy π_{θ}

Hyperparameter	Role in GRPO Training
batch_size	We will update old_policy to cur_policy after batch_size number of initial state were generated(See Line 3)
num_epochs	Number of outer iterations over the entire training process (Line 2: for iteration = 1, ..., I). Each epoch is a full pass over the data with multiple steps of policy updates.
num_episodes, G	Total number of initial state generated from which G episodes are generated for each initial state
μ (mu)	See Line 10
ϵ (epsilon)	clipping parameter in GRPO loss
β (beta)	Represents the weight on a KL-divergence penalty term in Overall GRPO loss term

Figure 2.1: Key hyperparameter for GRPO

2.2 Training LLMs on Custom Datasets

Training and fine-tuning large language models (LLMs) on custom datasets has become central to improving model performance on specific tasks. LLMs, such as GPT-4 and BERT, are typically pre-trained on massive corpora of general-purpose data.

However, fine-tuning allows adaptation to more specialized domains and can improve task-specific performance in tasks such as sentiment analysis, text summarization, and medical question answering. Fine-tuning LLMs typically involves supervised learning, where models are trained on labeled data corresponding to the target task. This process can be further enhanced with techniques like domain adaptation, in which the model is exposed to a smaller, task-relevant dataset to refine its general knowledge for specialized language patterns. Transfer learning is another powerful technique in which a pre-trained model is adapted to a new domain with minimal data. Fine-tuning can also be performed using unsupervised approaches, where models learn patterns or structures from unannotated data through methods such as self-supervised learning. When working with custom datasets, especially for NLP tasks like text summarization, data quality plays a crucial role. The choice of fine-tuning techniques—such as training from scratch, supervised fine-tuning (SFT), or reinforcement learning—must align with dataset size, task complexity, and available computational resources.

2.3 Training Pipeline and Impact of RL (GRPO) on DeepSeek R1 Training

DeepSeek released two models: DeepSeek R1-Zero and DeepSeek R1 on Jan 2025. To train DeepSeek R1-Zero, we used DeepSeek V3 as the base model. Skipping the supervised fine-tuning step, we trained it directly using the GRPO reinforcement learning algorithm (since it is faster and uses less memory) and employed rule-based rewards instead of human feedback rewards.

Note: For tasks such as mathematics and coding, rule-based rewards can suffice; therefore, DeepSeek R1-Zero was primarily trained on these tasks.

An “aha moment” occurs when the model begins the reasoning process and, halfway through, evaluates its initial approach and corrects it as needed. DeepSeek R1-Zero performed well and was comparable to OpenAI O1 on various tasks. However, by skipping the supervised fine-tuning step, its outputs suffered from poor readability.

2.3.1 DeepSeek R1-Zero Evaluation

Model	AIME 2024		MATH-500	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
OpenAI-o1-mini	63.6	80.0	90.0	60.0	53.8	1820
OpenAI-o1-0912	74.4	83.3	94.8	77.3	63.4	1843
DeepSeek-R1-Zero	71.0	86.7	95.9	73.3	50.0	1444

Table 2 | Comparison of DeepSeek-R1-Zero and OpenAI o1 models on reasoning-related benchmarks.

Figure 2.2: Performance evaluation of DeepSeek R1-Zero.

The DeepSeek R1 training pipeline consisted of four phases. Phase 1 (Cold-start): The DeepSeek V3 base model was supervised fine-tuned on a dataset collected from DeepSeek R1-Zero to improve its reasoning capabilities. Phase 2 (GRPO RL): Identical to the DeepSeek R1-Zero step, a GRPO reinforcement learning step on the Phase 1 checkpoint trained on a large amount of data for tasks such as coding and mathematics. Phase 3 (Rejection Sampling & Supervised Fine-Tuning): The Phase 2 checkpoint generated various samples; unreadable samples were removed using rejection sampling. Then the Phase 2 checkpoint was further supervised fine-tuned on the remaining samples.

Note: Rejection sampling approximates sampling from a complex distribution using a simpler proxy distribution. Samples drawn from the proxy distribution q are accepted if $\frac{p(x)}{q(x)}$ exceeds a threshold; otherwise, they are rejected, resulting in samples following the target distribution p .

Phase 4 (RL with Diverse Tasks): The Phase 3 checkpoint was further trained using RL on a diverse set of tasks, where rewards were computed either via rule-based approaches (for math and coding tasks) or via human feedback (for other tasks).

Subsequently, smaller models (e.g., 32 B parameters) were distilled from the samples generated in Phase 3. Finally, the model matched the performance of OpenAI’s flagship O1 model on major evaluation metrics.

2.3.2 DeepSeek R1 Evaluation

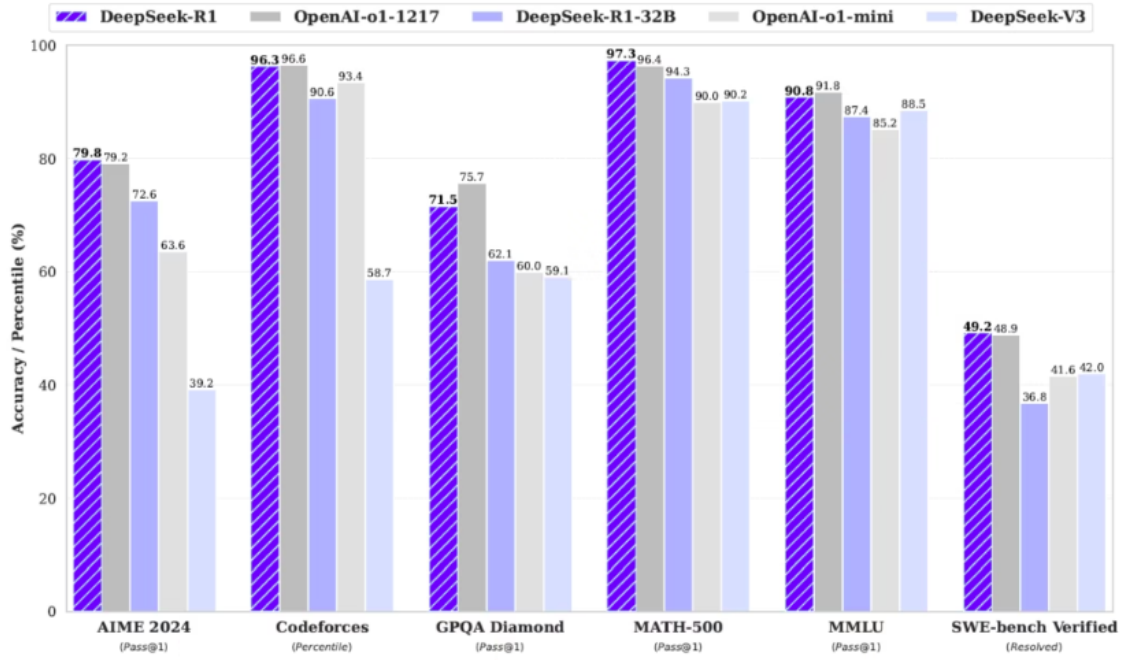


Figure 2.3: Performance evaluation of DeepSeek R1.

Chapter 3

Methodology

3.1 Text Generation and Summarization Methodology

3.1.1 Improving Text Generation with RL

Reinforcement Learning (RL) offers a powerful framework for enhancing generative text tasks such as question answering, summarization, and dialogue generation. Traditional supervised learning methods, while effective for many tasks, often struggle to capture complex human preferences and to produce high-quality, contextually relevant outputs. RL, and more specifically Reinforcement Learning with Human Feedback (RLHF)⁽⁴⁾, addresses this gap by enabling models to learn from dynamic feedback, thereby aligning their outputs more closely with human expectations.

In generative tasks, RLHF allows models to iteratively refine their responses based on feedback from human evaluators. Instead of training solely on fixed labels, RLHF trains the model using reward signals that reflect human judgments about output quality. This approach is particularly valuable in question answering or dialogue generation, where the “correct” output can be subjective and dependent on context, tone, or complexity.

By employing RL algorithms such as Proximal Policy Optimization (PPO)—known for its stability and scalability—RLHF guides generative models toward more coherent, informative, and contextually appropriate responses. Using the entropy term in the loss function, PPO balances exploration (trying different answers) and exploitation (refining successful answers), enabling models to improve both accuracy and alignment with human preferences through an iterative feedback loop.

3.1.2 RLHF Architecture: Why PPO and Not A2C?

In RLHF for generative text tasks, the choice of RL algorithm is critical. PPO is often preferred over A2C (Advantage Actor-Critic), A3C, and TRPO (Trust Region Policy Optimization) for the following reasons:

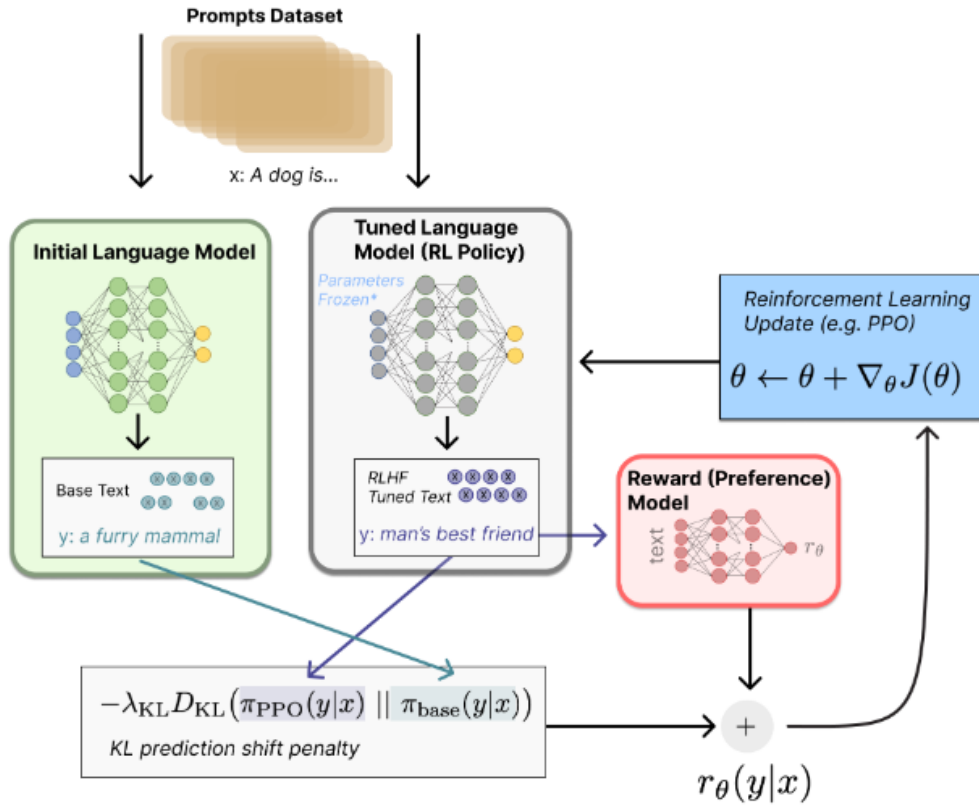


Figure 3.1: RLHF architecture for generative text generation

Stability and Efficiency: PPO’s clipped objective prevents excessively large policy updates, reducing training instability. A2C, by contrast, updates actor and critic simultaneously without such safeguards, making it prone to divergence in high-dimensional settings.

Scalability: PPO scales well to large models and complex environments typical of LLM fine-tuning. A3C’s parallel agents introduce overhead and potential instability, while TRPO’s second-order updates are computationally expensive.

Ease of Implementation: PPO relies on first-order optimization and simple clip-

ping, whereas TRPO requires complex constraint solving and Hessian-vector products.

Proven Performance in NLP: PPO has demonstrated strong results in language modeling, summarization, and dialogue, effectively balancing output diversity and quality.

Note: A2C offers simplicity but lacks the robust update mechanism of PPO, leading to unstable performance in sparse-reward environments like text generation, whereas A3C utilizes parallelism to speed up training but still suffers from PPO-level stability and requires significant engineering to coordinate multiple agents.

3.1.3 Our Task: Text Summarization

Text summarization in NLP aims to condense a document into a concise form while preserving its essential information. There are two main approaches:

a) Extractive Summarization selects key sentences or phrases directly from the source. It is computationally simpler and guarantees factual consistency but often yields disjointed or redundant summaries.

b) Abstractive Summarization generates new sentences that paraphrase the original text. Leveraging sequence-to-sequence models, transformers, and RLHF fine-tuning, abstractive methods produce more fluent and coherent summaries at the cost of greater model complexity.

In this work, we focus on abstractive summarization, aiming to generate contextually accurate and concise summaries that capture the essence of the source text.

3.1.4 Dataset: OpenAI/SummarizeFromFeedback

We use the **OpenAI/SummarizeFromFeedback** dataset, designed for RLHF-based summarization. It pairs documents with human-written summaries and includes human feedback—ratings and preferences—that guide RL fine-tuning.

3.1.4.1 Dataset Overview

The dataset contains:

- **Text:** Original documents to be summarized.

- **Summary:** Human-generated reference summaries.
- **Choices:** Alternative model outputs evaluated by annotators.
- **Ratings/Feedback:** Human judgments on relevance, fluency, and coherence.

info	summaries	choice
dict	list	int32
1 50.6%		
<pre>{ "id": "t3_34xale", "post": "My boyfriend and I are long distance. We have a trip planned this summer which involves me going over to him in the USA. This will be the second time I have actually been with him in person. I am flying from the UK with my mum to the east coast. The original plan was for me to fly over to my boyfriend in the west coast (my parents are holidaying on the east coast) but because my mum was freaking out so much about me going to meet my boyfriend i said we can all road trip there together. I even invited her on the trip with us. I have given her all of our dates so that she can travel around with us.\n\nThe plan was for me to stay on the 4th July and fly back on the 5th. Mum knew this. I told her I had booked a flight back already from the west coast to east coast (where she would pick me up and we would fly back to the UK together). She has gone mad at me because she can't believe I would book a flight when she told me she didn't want me flying on my own. At the time I had booked it she told me she wasn't gonna road trip with us. She knew the trip was happening.....how else was I to get home if I don't fly? \n\nI am fine flying on my own it doesn't bother me at all. I feel like I have done everything I can to make her feel comfortable with this trip and she is just trying to sabotage it. Thoughts??", "title": "Mother [51] not speaking to me [21] because of a trip I am planning", "subreddit": "relationships", "site": null, "article": null }</pre>	<pre>[{ "text": " Mum is mad at me for not flying on my own trip to meet my boyfriend.", "policy": "supl", "note": null }, { "text": " I have made sure my mother is comfortable with my boyfriend travelling on a trip and now my mother is mad because I booked it.", "policy": "supl", "note": null }]</pre>	1

Figure 3.2: Example schema from the OpenAI SummarizeFromFeedback dataset

3.1.5 Models

3.1.5.1 RL Algorithms Considered

We explored several RL methods:

- **TRPO:** Stable but computationally intense due to second-order updates.
- **DQN, DDPG, SAC:** Rely on replay buffers; less suited to sequential text outputs.
- **A2C:** Simple but unstable in sparse-reward, high-dimensional tasks.
- **A3C:** Parallelized but still less stable and more complex than PPO.

3.1.5.2 Why PPO?

PPO offers a balance of sample efficiency, stability, and computational cost. Its clipped objective constrains updates within a trust region, preventing large policy shifts that could degrade performance.

3.1.5.3 Computational Challenges and Optimization Strategies

Fine-tuning LLaMA-7B with PPO is resource-intensive. Loading two 7B-parameter models in 4-bit quantization requires approximately 7 GB of VRAM per model. To avoid CUDA out-of-memory errors on 16 GB VRAM, we:

- Reduced batch size to 1.
- Employed gradient accumulation to simulate larger batches.
- Excluded frozen parameters from the optimizer to conserve memory.

This approach helped manage the limited VRAM (16GB) available in the Kaggle notebook and provided stability during training.

3.1.5.4 Use of Smaller Models for Reward Classification

While the main model for text summarization requires a large-scale architecture, the reward model can often be smaller and still effectively perform classification tasks. For this, I employed the Gemma 2B model, which proved to be efficient for reward classification. Unlike the generative task, where large models are necessary for capturing the complexity of language, the reward classification task benefits from smaller models like Gemma 2B, which can handle the text classification aspect of RLHF efficiently.

3.1.5.5 Direct Preference Optimization (DPO)

Direct Preference Optimization (DPO) aligns model outputs directly with human preferences without a separate reward model, reducing computational overhead and enabling faster, resource-efficient fine-tuning (18).

3.2 Blackjack Environment

3.2.1 Game Setup

Blackjack is a two-party card game between a player and a dealer. Each receives two cards. The player aims for a hand value as close to 21 as possible without exceeding it. Number cards are worth their face value; face cards (J, Q, K) are worth 10; Aces count as 1 or 11, whichever is more advantageous.

3.2.2 Player Actions

- **Hit:** Draw one additional card.
- **Stick (Stand):** Keep the current hand and end the turn.
- **Double Down:** Double the bet, draw one card, then stick. Allowed only on the first move.
- **Surrender:** Forfeit half the bet and end the round. Allowed only on the first move.

3.2.3 Dealer's Turn

After the player sticks or surrenders, the dealer reveals the hidden card and:

- Hits on any total of 16 or less.
- Sticks on any total of 17 or higher.

3.2.4 Card Counting and Strategy

With a fixed number of decks and no reshuffle, players can track the ratio of high- to low-value cards to adjust decisions and bet sizes for an edge over the dealer.

3.2.5 Natural Blackjack

A *natural blackjack* occurs when the first two cards are an Ace and a 10-value card. It pays 3:2 and beats any dealer hand except another natural, which results in a push(i.e, player get's its bet back but no extra money).

3.2.6 Types of Blackjack Environments

3.2.6.1 Simple Blackjack Environment

- **Actions:** HIT, STICK
- **Implementation:** Gymnasium’s default Blackjack environment.

3.2.6.2 Extended Blackjack Environment

- **Actions:** HIT, STICK, DOUBLE DOWN, SURRENDER
- **Implementation:** Custom Gymnasium class supporting extra actions.

3.2.6.3 Extended Blackjack Environment with Card Counting

- **Actions:** As in the extended environment.
- **Decks:** Single deck, no reshuffle.
- **Implementation:** Tracks running card count.

3.2.7 Evaluation Metrics for Blackjack

- **Win%, Draw%, Loss%.** For N episodes and reward R_e in episode e :

$$\begin{aligned}\text{Win\%} &= \frac{|\{e : R_e > 1\}|}{N} \times 100\%, \\ \text{Draw\%} &= \frac{|\{e : R_e = 0\}|}{N} \times 100\%, \\ \text{Loss\%} &= \frac{|\{e : R_e < 0\}|}{N} \times 100\%.\end{aligned}$$

- **Average Reward per Bet.** With wager B_e in episode e :

$$\text{Avg. Reward per Bet} = \frac{\sum_{e=1}^N R_e}{\sum_{e=1}^N B_e} \times 100\%.$$

3.2.8 Training Setup

3.2.8.1 Simple Blackjack Environment

We trained both the Monte Carlo method and a PPO model on the simple environment.

3.2.8.2 Extended Blackjack Environment

We also trained the Monte Carlo method and a PPO model on the extended environment.

3.2.8.3 Extended Blackjack Environment with Card Counting

Hugging Face, “Monte Carlo vs Temporal-Difference Learning,” *Deep Reinforcement Learning Course*, 2023. [Online]. Available: <https://huggingface.co/learn/deep-rl-course/en/unit2/mc-vs-td>

Chapter 4

Results and Discussion

4.1 LLM Experiments

Code, results for all agents are available at:

<https://github.com/JaishreeramCoder/Masters-Thesis-Project1>

4.1.1 Training RLHF Model with GPT-2

In this experiment, we trained a Reinforcement Learning from Human Feedback (RLHF) setup using **GPT-2** as both the main model and the reward model. We employed the **SummarizeFromFeedback** dataset—comprising text inputs paired with human-annotated summary ratings—and optimized via PPO (Proximal Policy Optimization). After training, the average reward remained low, almost equal to 2, suggesting that GPT-2 with our current RLHF configuration requires either more refined hyperparameter tuning or an improved reward formulation.

4.1.2 Gemma-2B vs. Gemma-7B on Text Classification

We compared **Gemma-2B** and **Gemma-7B** on a subset of the IMDB Review Dataset for binary sentiment classification. Both models achieved within 2% of each other in accuracy and F1 score, indicating that for classification tasks the smaller 2 B model suffices. In contrast, for generative tasks such as summarization, Gemma-7B consistently outperformed Gemma-2B, confirming that larger models excel at generation.

4.1.3 Fine-Tuning Gemma-2B Reward Model & LLaMA-3.1 8B

Reward-Model Fine-Tuning (Gemma-2B) We fine-tuned Gemma-2B on a summarization-quality classification task for 5 epochs to serve as our reward predictor.

Supervised Fine-Tuning (LLaMA-3.1 8B) Using a 16 GB GPU, we ran 5 epochs of supervised fine-tuning on LLaMA-3.1 8B for summarization, achieving a Rouge-1 F1 of 18%.

RLHF Fine-Tuning (LLaMA-3.1 8B + Gemma-2B) We then applied PPO for 4 epochs using the fine-tuned Gemma-2B reward model. The average reward increased modestly over the supervised baseline, demonstrating that RLHF can further align output with human preferences.

DPO Comparison We also trained a Direct Preference Optimization (DPO) model. It required under 30% of PPO’s training time yet achieved similar loss, making DPO a compelling, compute-efficient alternative.

4.2 Results for the Blackjack Environment

Code, results, and trained model weights for all agents are available at <https://github.com/JaishreeramCoder/blackjack/>.

4.2.1 Simple Blackjack Environment

```
class Blackjack(gym.Env):  
    """  
    State Space:  
    - player_sum: Integer [0,32], sum of player's hand  
    - dealer_visible_card: Integer [1,10], dealer's one visible card  
    - usable_ace: Boolean, whether player has a usable ace  
      (counts as 11 without busting)  
  
    Action Space:  
    Discrete(2):  
    0: STICK (stop taking cards)  
    1: HIT (draw another card)
```

Reward Structure:

*+1 Player wins
0 Draw
-1 Player loses*

Additional Rules:

*- Natural blackjack (21 with first two cards) gives a reward
of +1.5 if the "natural" flag is set to True in the env config.*

"""

... (rest of the implementation)

4.2.1.1 Monte Carlo Method

The Monte Carlo agent was trained with the following hyperparameters:

$$\gamma = 1, \quad \varepsilon\text{-greedy with } \varepsilon = 0.1, \quad \text{Episodes} = 10,000,000.$$

The model got flattened after almost 4e6 episodes(see Fig. 4.1), and during evaluation (see Fig. 4.2) gave average reward per bet of -4.68%, and win% of 43.25%. The strategy chart is available at:(see Fig. 4.3)

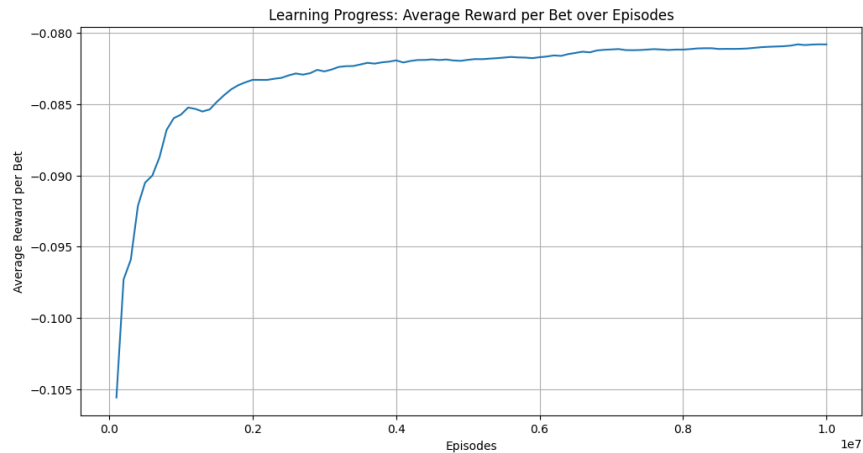


Figure 4.1: Average reward per bet during training of the Monte Carlo agent.

Evaluation Results

Metric	Value
Total Episodes	100000
Win %	43.25%
Loss %	47.93%
Draw %	8.82%
Average Reward per Bet	-4.68%

Figure 4.2: Evaluation metrics for the Monte Carlo agent after training.

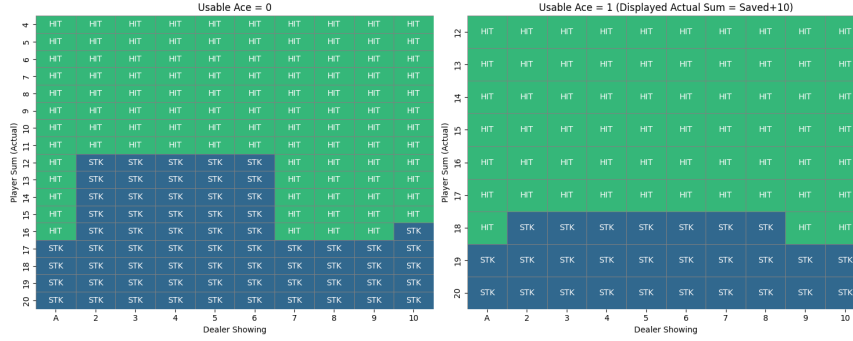


Figure 4.3: Learned strategy heatmap (policy chart) for the Monte Carlo agent in the simple environment.

4.2.1.2 Proximal Policy Optimization (PPO)

Key Hyperparameters in PPO (Stable Baselines) The Proximal Policy Optimization (PPO) implementation in Stable Baselines uses several important hyperparameters:

- **Timestep vs. Episode.** A *timestep* is a single interaction between the agent and environment. An *episode* consists of all timesteps from the start state until termination.
- **n_steps** (default: 2048). Number of timesteps collected *before* each policy update.
- **batch_size** (default: 64). The *n_steps* collected transitions are split into mini-batches of this size. For example, with *n_steps* = 2048 and *batch_size*=64,

you get

$$\frac{2048}{64} = 32 \text{ minibatches per update.}$$

- **n_epochs** (default: 10). Number of passes (epochs) over the entire set of n_steps samples during each update. One full update therefore, executes

$$n_epochs \times \frac{n_steps}{batch_size} = 10 \times 32 = 320$$

gradient steps in total.

Example calculation. Over 10^7 total timesteps:

$$\begin{aligned} \text{Number of policy updates} &= \frac{10^7}{2048} \approx 4883, \\ \text{Total gradient steps} &= 4883 \times (10 \times 32) \approx 1.56 \times 10^6. \end{aligned}$$

The PPO agent was trained with the following hyperparameters:

$$\gamma = 1.0, \quad n_steps = 2048, \quad batch_size = 64, \quad ent_coef = 0.01,$$

learning_rate: cosine schedule with $\min = 1 \times 10^{-6}$, $\max = 1 \times 10^{-4}$, $timesteps = 20,000,000$.

During evaluation (see Fig. 4.4) model gave average reward per bet of -4.57%, and win% of 43.12%.

Evaluation Results

Metric	Value
Total Episodes	100000
Win %	43.12%
Loss %	47.70%
Draw %	9.18%
Average Reward per Bet	-4.57%

Figure 4.4: Evaluation metrics for the PPO agent in the simple Blackjack environment.

4.2.2 Extended Blackjack Environment

```
class BlackjackEnvInfinite(gym.Env):
    """
    Custom Blackjack environment with an infinite deck.

    State (MultiDiscrete):
        [player_sum, dealer_card, usable_ace, first_move]

    - player_sum: Sum of player's hand (0 to 31)
    - dealer_card: Dealer's face-up card (1-10; note: 0 is unused)
    - usable_ace: 0 (False) or 1 (True)
    - first_move: 1 if it's the first decision, else 0

    Actions (Discrete(4)):
        0: HIT          Request another card.
        1: STK          Stand.
        2: DBL          Double Down (only on first move).
        3: SUR          Surrender (only on first move).

    Reward Structure:
    - Blackjack pays 3:2 (payout_blackjack=1.5) only if the player has
      blackjack.
    - Regular win pays 1:1.
    - Push returns 0.
    - Loss costs the bet.
    - Surrender returns -0.5 times the base bet.
    - Double Down outcomes are scaled (bet multiplied by 2).
    """
    # ... (rest of the implementation)
```

4.2.2.1 Monte Carlo Method

The Monte Carlo agent was trained with the same hyperparameters as in the simple environment:

$$\gamma = 1, \quad \varepsilon\text{-greedy with } \varepsilon = 0.1, \quad \text{Episodes} = 10,000,000.$$

The model got flattened after almost 8e6 episodes, and during evaluation (see Fig. 4.6) gave average reward per bet of -0.45%, and win% of 41.12%. The average reward per bet improved significantly in the extended environment. The strategy chart is available at:(see Fig. 4.7)

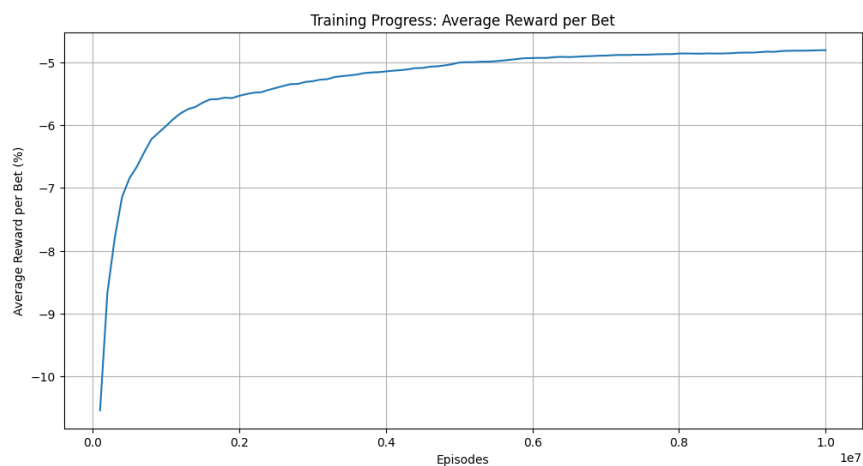


Figure 4.5: Average reward per bet during training of the Monte Carlo agent in the extended environment.

Evaluation Results

Metric	Value
Total Episodes	100000
Win %	41.12%
Loss %	50.84%
Draw %	8.04%
Average Reward per Bet	-0.45%

Figure 4.6: Evaluation metrics for the Monte Carlo agent in the extended environment.

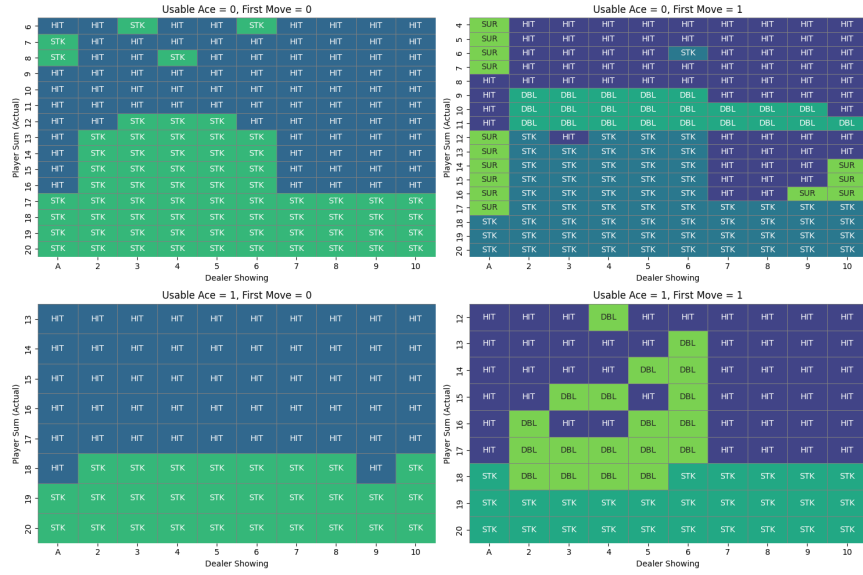


Figure 4.7: Learned strategy heatmap for the Monte Carlo agent in the extended environment.

4.2.2.2 Proximal Policy Optimization (PPO)

The PPO agent was trained with the following hyperparameters:

$\gamma = 1.0$, $n_steps = 2048$, $batch_size = 64$, $ent_coef = 0.01$,

learning_rate: cosine schedule with $\min = 1 \times 10^{-6}$, $\max = 1 \times 10^{-4}$, $timesteps = 20,000,000$.

During evaluation (see Fig. 4.9) model gave average reward per bet of -0.99%, and win% of 38.47%. The average reward per bet has improved significantly in the extended environment.

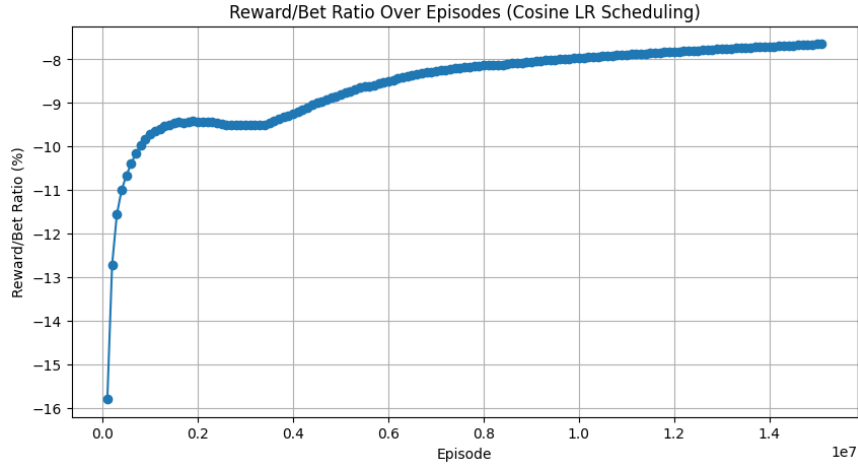


Figure 4.8: Average reward per bet during training of the PPO agent in the extended environment.

Evaluation Results

Metric	Value
Total Episodes	100000
Win %	38.47%
Loss %	54.06%
Draw %	7.47%
Average Reward per Bet	-0.99%

Figure 4.9: Evaluation metrics for the PPO agent in the extended environment.

4.2.2.3 Important Insights

- **Simple Environment:** The average reward per bet and win percentage for the Monte Carlo and PPO models are nearly identical. The Monte Carlo method converged in approximately 30 minutes (training curves flattened), whereas PPO required around 11 hours and showed no improvement after an additional 20 000 000 timesteps (cosine LR from 10^{-6} to 10^{-7}). Thus, for the simple environment, traditional Monte Carlo remains the most efficient choice.
- **Reward Magnitude:** Both models achieved an average reward per bet of about -4.5% in the simple environment. This lower performance stems from the absence of $1.5\times$ (natural blackjack) and $2\times$ (double down) payouts. Extended environments allowing these payouts yield substantially higher returns, and further training did not improve results in the simple setup.

4.2.3 Extended Blackjack Environment with Card Counting

```
class BlackjackEnvCountingFirstMove(gym.Env):  
    """  
    Custom Blackjack environment with card counting and a  
    first-move flag.  
  
    State (MultiDiscrete):  
    [player_sum, usable_ace, dealer_card, is_first_move,  
     cnt_A, cnt_2, ..., cnt_9, cnt_10group]  
  
    - player_sum: Sum of player's hand (0 to 31)  
    - usable_ace: 0 (no usable ace) or 1 (usable ace exists)  
    - dealer_card: Dealer's face-up card  
                  (1 for Ace, 2--10 for number cards)  
    - is_first_move: 1 if first decision of episode, else 0  
    - cnt_A: Count for Ace (1)  
    - cnt_2 to cnt_9: Count for cards 2 through 9  
    - cnt_10group: Count for 10, Jack, Queen, King (10)  
  
    Actions (Discrete(4)):  
    0: HIT          Request another card.  
    1: STK          Stand.  
    2: DBL          Double Down (only on first move).  
    3: SUR          Surrender (only on first move).
```

After first move, only HIT (0) and STK (1) are allowed.

Reward Structure:

- Blackjack pays 3:2 (payout_blackjack=1.5) only if the player has blackjack.*
- Regular win pays 1:1.*
- Push returns 0.*
- Loss costs the bet.*
- Surrender returns -0.5 times the base bet.*
- Double Down outcomes are scaled (bet *2).*

"""

... (rest of the implementation)

4.2.3.1 Why Deep RL-Based Methods Are Required?

The state space of the card-counting environment grows combinatorially:

```
State space size = 32 × 2 × 10 × 2 × 59 × 17
                  = 32 × 2 × 10 × 2 × 33,203,125
                  = 4,250,000,000
42.5 billion unique states
```

- Vastly larger than the 1,408 states of the extended env.
- Tabular methods (Monte Carlo, Q-Learning, TD) cannot store/update tens of billions of state-action pairs.
- Deep RL (DQN(2), PPO, GRPO) uses neural nets to approximate value/policy, enabling generalization to unseen states.
- Function approximators let the agent learn strategies without visiting every state explicitly.

4.2.3.2 Proximal Policy Optimization (PPO)

When first evaluated on 100,000 episodes in the card-counting env, vanilla PPO achieved only a 32% win rate and an average reward per bet of -9% (i.e. a \$100 bet returned \$91), far below the > 40% win rates of other methods.¹

Potential Issues and Remedies

¹<https://www.kaggle.com/code/jaishreeramcoder/ppo-model-blackjack>

1. **Learning Rate Instability:** Default LR 3×10^{-4} gave rapid early gains but fluctuated later. *Solution:* Switched to cosine LR $1 \times 10^{-4} \rightarrow 1 \times 10^{-6}$, boosting avg. reward to +2.36%.
2. **Network Capacity:** Default PPO nets (2×64) were increased to 3 layers (128,64,32) but no further gains meaning the capacity was sufficient.

Extended Training and Convergence

- *Phase 1:* 10 000 000 timesteps (70M episodes), cosine LR $1 \times 10^{-4} \rightarrow 1 \times 10^{-6}$. ~7 hours; avg. reward still rising (see Figure 4.10).
- *Phase 2:* +20 000 000 timesteps, LR $1 \times 10^{-6} \rightarrow 1 \times 10^{-7}$. Further training saw no improvement $\rightarrow$ converged.

Additional Observations

- *Timesteps/Episode:* Tracking average timesteps per episode could reveal whether the agent adopts a defensive (more STK) or aggressive (more HIT) style; this analysis remains future work
- *GPU vs. CPU:* With a relatively small two-layer network, GPU acceleration offered no appreciable speedup or performance benefit
- *Catastrophic Forgetting:* Over many episodes, early-learned rewards can be forgotten, akin to overfitting in supervised learning.

Code:

- Phase 1: [Phase 1 Model Training](#)
- Phase 2: [Phase 2 Model Training](#)

Phase 1 Training Results The PPO agent was trained with:

$\gamma = 0.99$, $n_steps = 2048$, $batch_size = 64$, $ent_coef = 0.01$,
learning_rate: cosine schedule with $min = 1 \times 10^{-6}$, $max = 1 \times 10^{-4}$, $timesteps = 10,000,000$.

The model’s training plot didn’t get flattened after 1e7 timesteps, Therefore, further training was done in phase 2. During evaluation (see Fig. 4.11), it gave average reward per bet of 1.77%, and win% of 41.25%. The average reward per bet improved significantly in the extended environment.

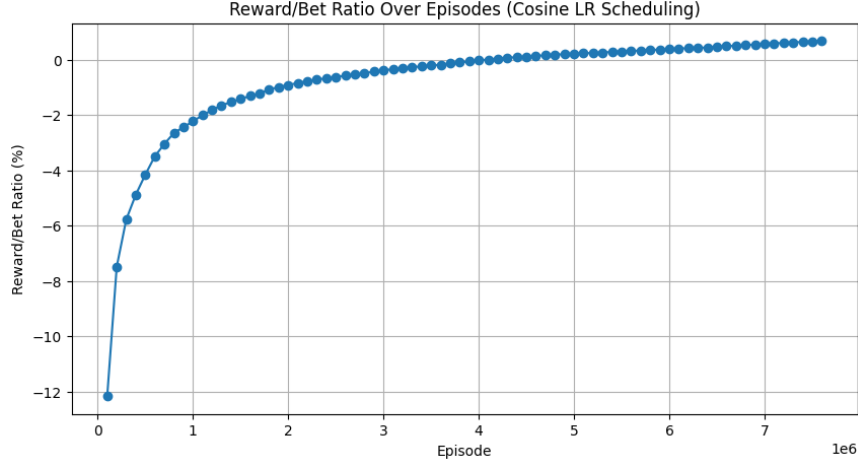


Figure 4.10: Phase 1: Avg. reward per bet during PPO training (10 M timesteps).

Evaluation Results

Metric	Value
Total Episodes	100000
Win %	41.25%
Loss %	51.49%
Draw %	7.26%
Average Reward per Bet	1.77%

Figure 4.11: Phase 1: Evaluation metrics after 10 M timesteps.

Phase 2 (Final) Training Results The PPO agent was further trained with:

$\gamma = 0.99$, $n_steps = 2048$, $batch_size = 64$, $ent_coef = 0.01$,

$learning_rate$: cosine schedule with $min = 1 \times 10^{-7}$, $max = 1 \times 10^{-6}$, $timesteps = 20,000,000$.

The training plot(see Fig. 4.12) didn't get flattened after $2e7$ timesteps, But when we further trained the model on another $2e7$ timesteps with cosine learning rate between $1e-7$ to $1e-8$, we observed no further improvement in performance, showcasing the model has converged. During evaluation (see Fig. 4.13), it gave average reward per bet of 2.36%, and win% of 41.58%. The average reward per bet improved significantly in the extended environment. Positive reward is very significant because that means that if we play a sufficiently large number of games, then on every 100 Rupees bet we will get 102.36 Rupees back in total.

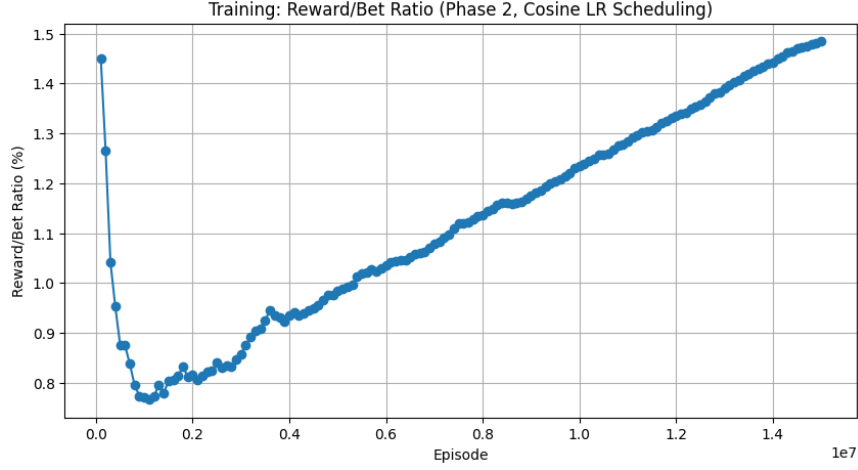


Figure 4.12: Phase 2: Avg. reward per bet during PPO training (+20 M timesteps).

Evaluation Results

Metric	Value
Total Episodes	100000
Win %	41.58%
Loss %	51.08%
Draw %	7.34%
Average Reward per Bet	2.36%

Figure 4.13: Phase 2: Evaluation metrics after total of 30 M timesteps.

4.2.3.3 Group Relative Policy Optimization Training (GRPO)

For a Gymnasium environment like Blackjack, there’s no off-the-shelf GRPO implementation, so we built it from scratch.

Training hyperparameters (initial):

$$\begin{aligned}
 \text{batch_size} &= 64, \text{ num_epochs} = 100, \text{ num_episodes} = 100,000, G = 8, \\
 \mu &= 10, \varepsilon = 0.2, \beta = 0.04, \\
 \text{learning rate: cosine } &1 \times 10^{-6} \rightarrow 1 \times 10^{-4} \quad (4.1)
 \end{aligned}$$

Training GRPO in the card-counting environment initially proved very time-consuming due to setting $\mu = 10$, which incurred ten backpropagation steps per

episode.

Despite achieving decent enough win rates, the GRPO agent’s average reward per bet was notably poor. We investigated the reasons as follows:

1. **Natural Blackjack Inclusion:** I suspected natural blackjacks were omitted from `reward_history`, since I faced a similar situation while training the PPO model. After recomputing, we confirmed they were correctly included.
2. **Reward Distribution Analysis:** Compared PPO vs. GRPO distributions (Figure 4.14). GRPO did only 5 double-downs per 100 000 vs. PPO’s 11 000,

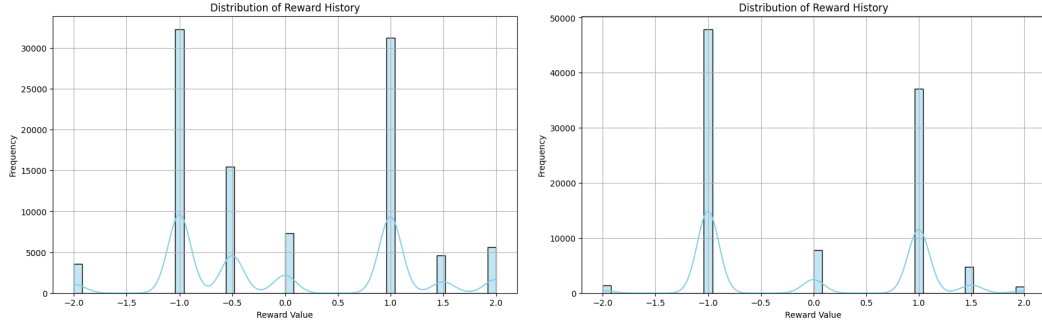


Figure 4.14: Reward distributions over 100 000 episodes: PPO (left) vs. GRPO (right).

explaining its lower avg. payoff despite similar win rates.

3. **Hyperparameter Tuning:** Reduced $\mu \rightarrow 2$, $G = 8$, episodes $\rightarrow 50,000$, epochs $\rightarrow 5$, improving performance slightly.
4. **Entropy Regularization:** Noting the absence of entropy in GRPO’s loss, we integrated an entropy bonus guided by [this paper](#), which modestly enhanced exploration.
5. **Action Masking:** Since illegal Double Down/Surrender after the first move incurred -1 , we added masks in the policy logits to disable these actions when its not the first move.
6. **DAPO Loss Integration:** Replaced GRPO objective with DAPO:

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\} \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{\sum_i |o_i|} \sum_{i,t} \min(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \hat{A}_{i,t}) \right],$$

$$\text{s.t. } 0 < |\{o_i \mid \text{is_equivalent}(a, o_i)\}| < G,$$

DAPO paper highlights the limitations of the GRPO algorithm’s sample-level loss in long-CoT scenarios, where longer responses are under-penalized, leading to poorer quality outputs. The proposed solution is a token-level normalization, which better handles longer sequences by assigning more balanced rewards to individual tokens, regardless of response length

Final GRPO Training Results: Finally, the best results were obtained using GRPO with DAPO loss having action masking(6) and following hyperparamters:

$$\begin{aligned} \text{batch_size} &= 64, \text{ num_epochs} = 10, \text{ num_episodes} = 20,000,000, G = 8, \\ \mu &= 2, \varepsilon = 0.2, \beta = 0.04, \\ \text{learning rate: cosine } &1 \times 10^{-7} \rightarrow 1 \times 10^{-4} \quad (4.2) \end{aligned}$$

We trained the DRPO model on 1e8 episodes(see Fig. 4.16). Even though its graph didn’t flatten, subsequent training on an additional 2e8 episodes didn’t improve the model’s performance. During evaluation, it hit a 44.39% win rate, though its average reward per bet remained below PPO.(see Fig. 4.17). Further, the reward distribution(see Fig. 4.15) of DRPO was more diverse compared to the original GRPO, showcasing that DRPO has enabled exploration in the model.

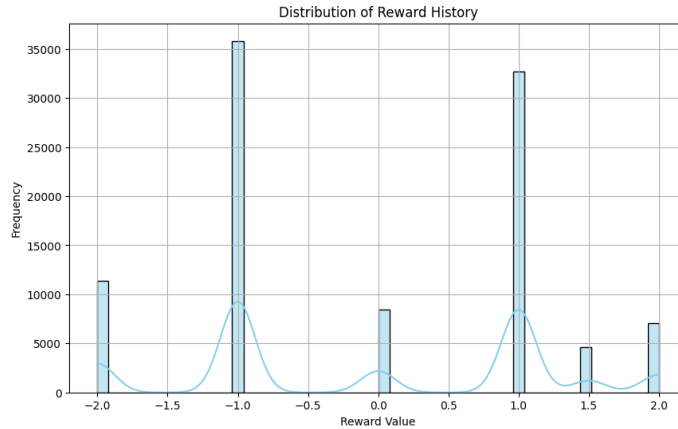


Figure 4.15: Reward distribution for DAPO-enhanced GRPO (`drpo_distribution`).

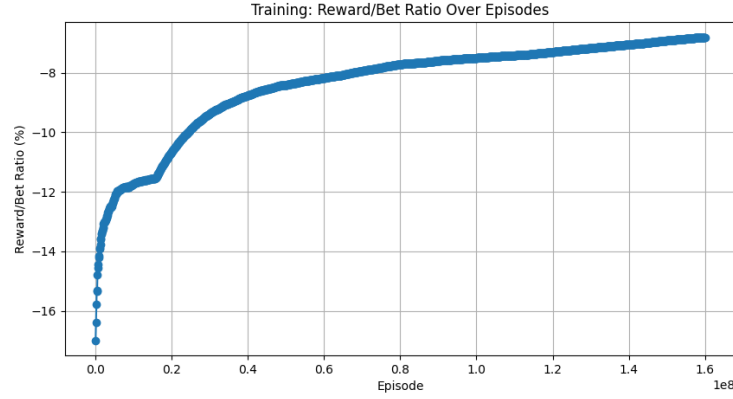


Figure 4.16: Average reward per bet during training of the final GRPO(DAPO) agent in the extended environment.

Evaluation Results

Metric	Value
Total Episodes	100000
Win %	44.39%
Loss %	47.20%
Draw %	8.41%
Average Reward per Bet	-3.03%

Figure 4.17: Evaluation metrics for the final GRPO(DAPO) agent in the extended environment.

4.2.3.4 Saving the Trained Models

- **Monte Carlo:** Saved $V(s)$ as a `.pickle` file. And during inference, the best action was found using: $a^* = \arg \max_a \sum_{s'} P(s'|s, a) V(s')$.
- **PPO:** Used SB3's `model.save("ppo_blackjack.zip")`, bundling policy, optimizer state, and hyperparams. During inference model is loaded using the `model.load()` function.
- **GRPO:** Used `torch.save(model.state_dict(), "grpo_policy.pth")` to save the model. During inference reloaded with `model.load_state_dict(...)`.

Chapter 5

Web Application for the Extended Blackjack Environment with Card Counting

For both the simple and extended Blackjack environments, the relatively small number of unique game states made it practical to generate complete strategy charts. These charts provide players with a straightforward reference for determining the optimal action—hit, stand, double down, or surrender based on the current state of the game. Such tools are especially helpful for new players looking to quickly learn and apply basic strategy.

However, the situation is markedly different in the card-counting Blackjack environment. Due to the introduction of additional variables to store the history of cards selected, the number of possible game states increases dramatically—estimated at approximately 42.8 billion unique combinations. This massive state space renders the creation of a comprehensive, static strategy chart infeasible, both in terms of data storage and user usability.

To address this challenge, we developed a dynamic, web-based assistant designed to provide real-time strategic guidance. Instead of consulting a pre-generated chart, players input their current state—such as their hand, the dealer’s visible card, and the running count—and receive the optimal action based on the underlying strategy model. This tool leverages algorithmic decision-making and pre-trained policies to compute recommendations on the fly, making it a practical and powerful resource for both casual players and serious card counters aiming to maximize their advantage.

Access the live demo at <https://blackjack-beta-kohl.vercel.app/>.

5.1 User Workflow

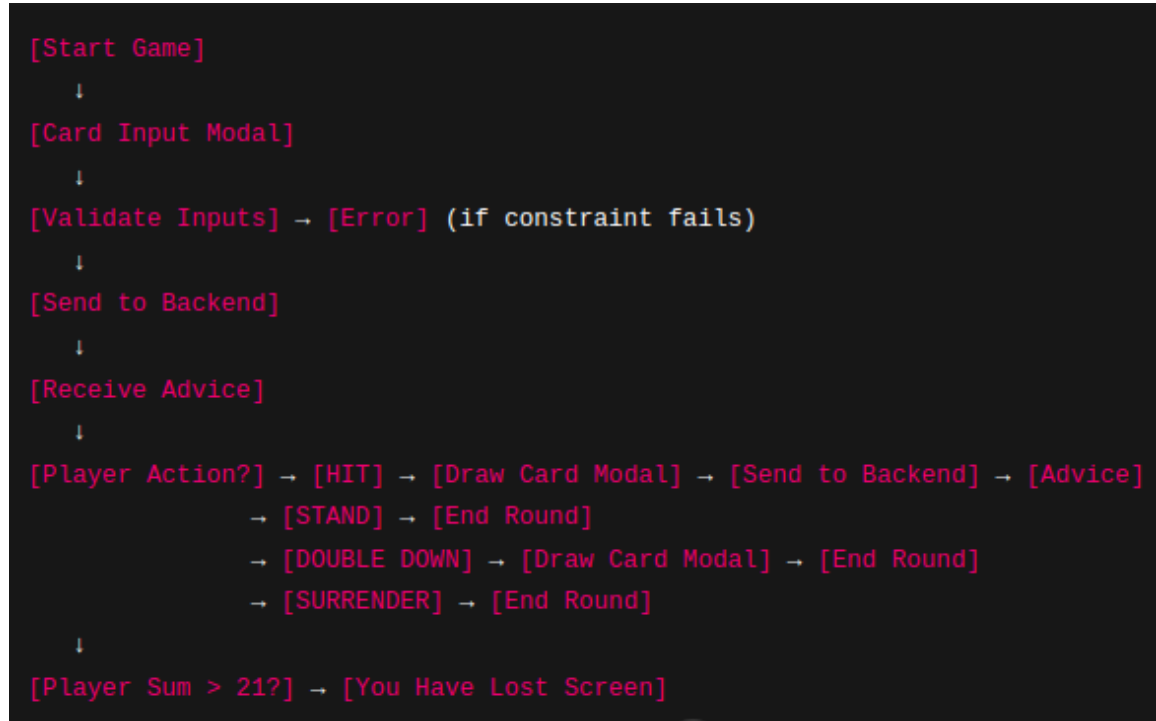


Figure 5.1: Flow diagram.

1. Click the **Start Game** button.
2. A modal appears to enter(see Fig. 5.2):
 - Dealer’s visible card.
 - Player’s current cards.

The modal enforces game constraints (e.g., no more than four of the same card value per deck).

3. Upon submission, the current state is sent via HTTP to the FastAPI backend, which returns the best action based on the loaded model. (see Fig. 5.3) If the player’s sum exceeds 21, a “You have lost” screen is displayed(see Fig. 5.4):
4. The player selects the advised action, subject to:

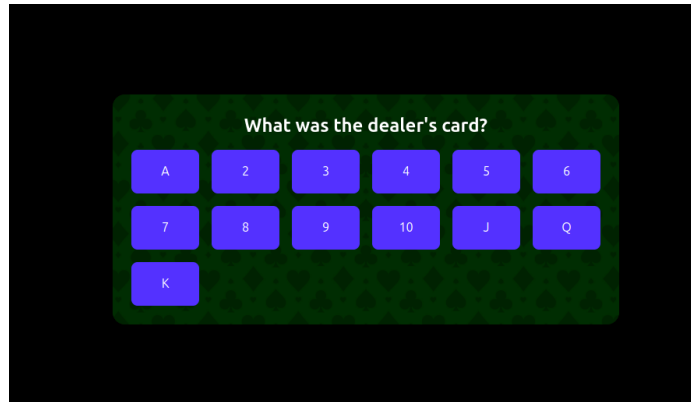


Figure 5.2: Modal for selecting dealer and player cards.

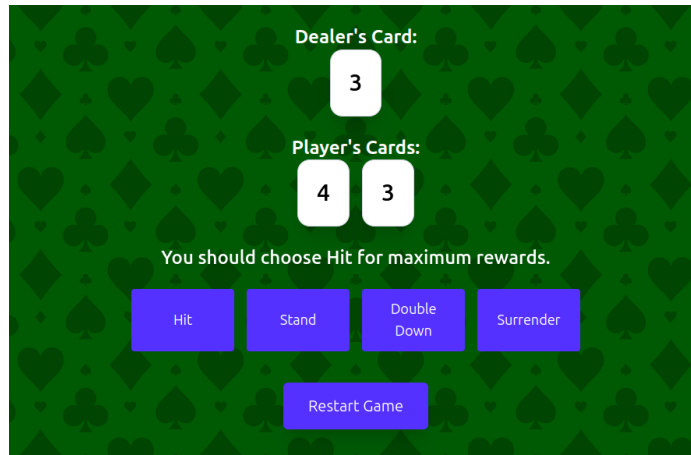


Figure 5.3: Main interface displaying game state and advice.

- **HIT:** A new modal prompts the player to choose the drawn card; advice updates accordingly.
- **STAND:** No further advice is provided.
- **DOUBLE DOWN:** A final modal to choose the drawn card; then advice ends.
- **SURRENDER:** The round ends immediately.

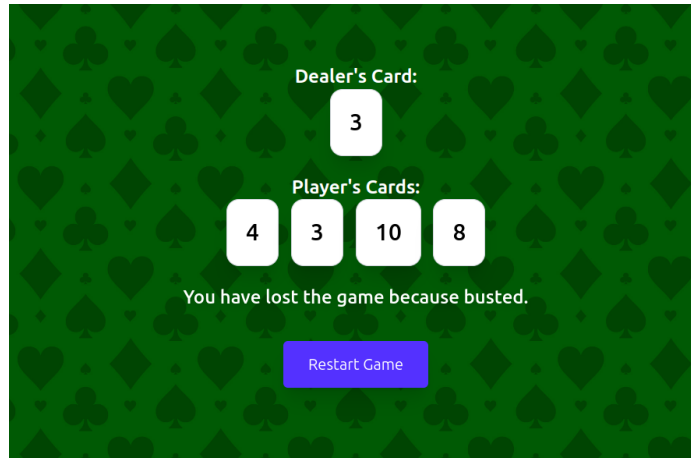


Figure 5.4: “Lost game” message when the player busts.

5.2 Implementation Details

- **Frontend:** Built with React and styled using Tailwind CSS v4.0.
- **Backend:** Developed with FastAPI.
- **Deployment:**
 - Frontend hosted on Vercel.
 - Backend hosted on Hugging Face Spaces (16 GB RAM free tier), since Render’s 512 MB limit was insufficient for loading the 1 GB+ model.
- **Styling Fixes:** Some Tailwind classes did not apply in production; custom utility classes were created to restore desired styles.
- **Cold Start Delay:** Since I have deployed the backend on Hugging Face Spaces’ free tier, it takes a few minutes to reload the model into the server’s RAM. This is because after a few hours of inactivity, the backend server removes the code from RAM.

Chapter 6

Conclusion

This work has investigated the application of Reinforcement Learning with Human Feedback (RLHF) and advanced policy-optimization techniques across two distinct domains: language model fine-tuning and sequential decision problems in games. In the context of text generation and summarization, we demonstrated that leveraging a reward model aligned with the base LLM (e.g., Gemma-7B) significantly enhances output quality when sufficient compute is available, while smaller architectures (e.g., Gemma-2B) offer a practical efficiency–performance trade-off for classification tasks. We further showed that Direct Preference Optimization (DPO) provides a computationally lighter alternative to Proximal Policy Optimization (PPO) without sacrificing alignment performance, and we stressed the importance of versioning rapidly evolving RLHF libraries—such as TRL—to ensure experimental reproducibility.

Turning to sequential decision environments, our experiments revealed that traditional Monte Carlo methods and PPO achieve comparable win rates and rewards in simple Blackjack settings, with Monte Carlo converging in minutes versus hours for PPO. When extending the environment to include card counting, we confronted the lack of off-the-shelf GRPO implementations for Gymnasium environments, prompting a custom implementation. Our GRPO agent, trained from scratch with a Decoupled Clip and Dynamic sAmpling Policy Optimization (DAPO) variant, attained the highest win rate, whereas the stepwise PPO from Stable Baselines excelled in average reward per bet. We further observed that, like in supervised learning, training the model on too many iterations can result in overfitting. Similarly, for reinforcement learning, if we train the complex state model on too many episodes, it may forget the information it learned in the earlier episodes.

Overall, RLHF and policy-optimization methods show strong potential for tailoring model behavior to human preferences and domain-specific objectives. Future work should carefully balance resource constraints, task complexity, and library version control when selecting methodologies for both LLM fine-tuning and control problems.

Chapter 7

Future Work

Looking ahead, we see four key avenues for extending this work. First, we will enhance GRPO by shifting from episode-level to step-level updates, mirroring the per-timestep refinements in Stable Baselines’ PPO. This is why the stable baseline’s PPO got a better average reward per bet. We will be required to build a new training pipeline for this, since the original pipeline suggested in the GRPO paper is for episode-level updates.

Second, we will implement Implicit Language Q-Learning (ILQL)(5) in Blackjack, including card-counting variants. By benchmarking ILQL against our existing Monte Carlo, PPO, GRPO, and DAPO baselines, we’ll assess whether its implicit policy structure better captures the nuanced state features critical to optimal play.

Third, to stress-test these methods in a truly cooperative, multi-agent context, we will adapt all four algorithms to Bridge. Designing suitable reward schemes and partner conventions, we’ll investigate how group-relative updates, and implicit versus explicit value estimation scale to the four-player, partially observable environment of contract bridge.

Finally, we will move beyond games into language, training both GRPO and ILQL on text summarization tasks using the trl and trlX libraries. Treating each generated token sequence as an episode and employing human-aligned quality metrics as rewards, this work will explore whether group-relative and implicit Q-learning frameworks can yield more coherent, high-quality summaries than current policy-gradient approaches.

Bibliography

- [1] PPO — Stable Baselines3 2.6.1a0 documentation. (n.d.). <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html>
- [2] Jha, S. (2020, July 5). *Deep Q Learning: A critique - Shubham Jha - Medium*. Medium. <https://medium.com/@jha1shubham/deep-q-learning-a-critique-ca4cbf92b8e5>
- [3] DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., & Zhang, Z. (2025). *DeepSeek-R1: Incentivizing reasoning capability in LLMs via Reinforcement Learning*. arXiv preprint arXiv:2501.12948. Retrieved from <https://arxiv.org/abs/2501.12948>
- [4] Illustrating Reinforcement Learning from Human Feedback (RLHF). (2001, March 25). <https://huggingface.co/blog/rlhf>
- [5] Snell, C., Kostrikov, I., Su, Y., Yang, M., & Levine, S. (2022, June 5). *Offline RL for Natural Language Generation with Implicit Language Q Learning*. arXiv.org. <https://arxiv.org/abs/2206.11871>
- [6] DAPO: An Open-Source LLM Reinforcement Learning System at scale. (n.d.). <https://arxiv.org/html/2503.14476v1>
- [7] F. Meng, Z. Yao, and M. Zhang, “TransMLA: Multi-Head Latent Attention Is All You Need,” *arXiv preprint arXiv:2502.07864*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.07864>
- [8] Artetxe, M., Bhosale, S., Goyal, N., Mihaylov, T., Ott, M., Shleifer, S., Lin, X. V., Du, J., Iyer, S., Pasunuru, R., Anantharaman, G., Li, X., Chen, S., Akin, H., Baines, M., Martin, L., Zhou, X., Koura,

- P. S., O'Horo, B., . . . Stoyanov, V. (2021, December 20). *Efficient Large Scale Language Modeling with Mixtures of Experts*. arXiv.org. [https://arxiv.org/abs/2112.10684#:~:text=Mixture%20of%20Experts%20layers%20\(MoEs,publicly%20available%20for%20research%20use](https://arxiv.org/abs/2112.10684#:~:text=Mixture%20of%20Experts%20layers%20(MoEs,publicly%20available%20for%20research%20use).
- [9] Vink, S. (2023b, May 22). *Temporal Difference Learning: SARSA vs Q-Learning - CodeX - Medium*. Medium. <https://medium.com/codex/temporal-difference-learning-sarsa-vs-q-learning-c367934b8bcc>
- [10] Vink, S. (2023c, May 22). *Temporal Difference Learning: SARSA vs Q-Learning - CodeX - Medium*. Medium. <https://medium.com/codex/temporal-difference-learning-sarsa-vs-q-learning-c367934b8bcc>
- [11] Fan, J., Wang, Z., Xie, Y., & Yang, Z. (2019, January 1). *A theoretical analysis of deep Q-Learning*. arXiv.org. <https://arxiv.org/abs/1901.00137>
- [12] GeeksforGeeks. (2025, February 26). *REINFORCE algorithm*. GeeksforGeeks. <https://www.geeksforgeeks.org/reinforce-algorithm>
- [13] Kargar, I. (2024, December 14). *RL Series-A2C and A3C - Isaac Kargar - Medium*. Medium. <https://kargarisaac.medium.com/rl-series-a2c-and-a3c-in-pytorch-6e9edf5c8788>
- [14] Deep Deterministic Policy Gradient — Spinning Up documentation. (n.d.). <https://spinningup.openai.com/en/latest/algorithms/ddpg.html>
- [15] Schulman, J., Levine, S., Moritz, P., Jordan, M. I., & Abbeel, P. (2015, February 19). *Trust Region Policy optimization*. arXiv.org. <https://arxiv.org/abs/1502.05477>
- [16] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017, July 20). *Proximal Policy optimization Algorithms*. arXiv.org. <https://arxiv.org/abs/1707.06347>
- [17] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., & Guo, D. (2024, February 5). *DeepSeekMath: Pushing the limits of mathematical reasoning in open language models*. arXiv.org. <https://arxiv.org/abs/2402.03300>

- [18] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2023, May 29). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. arXiv.org. <https://arxiv.org/abs/2305.18290>